

02393 Programming in C++

Module 10: Recursive Programming

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Recursion
- Complexity
- Lab

A recap from the previous lectures

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`, `class`, `this`
- Expressions
 - some numeric and boolean operators and math functions, conditional expressions
- Statements
 - `if`, `while`, `for`, `switch`

Recursion

- Recursive programming is a solution technique that solves large problems by reducing them to smaller problems of the same form
 - It is crucial that both the large and smaller problems have the same form!
 - So we can use the same technique (and the same code!) to solve both
- Why might recursion seem... weird?
 - Recursion requires a form of mathematical inductive reasoning
 - Recursion is more abstract than programming concepts having a mechanical intuition
 - loop: repeat an action several times
 - if-then-else: check a condition, make a decision
 - recursion: ???

Recursion

```
1 unsigned int factorial = 1;
2   for (unsigned int i = n; i > 0; i --) {
3       factorial = factorial * i;
4   }
5
6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }
```

- You may have already seen recursion in mathematical definitions. E.g., the factorial:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot ((n-1)!) & \text{otherwise} \end{cases}$$

- We can implement the factorial in C++ with an iterative approach

Recursion

```
1 unsigned int factorial = 1;
2 for (unsigned int i = n; i > 0; i --) {
3     factorial = factorial * i;
4 }
5
6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }
```

- You may have already seen recursion in mathematical definitions. E.g., the factorial:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot ((n-1)!) & \text{otherwise} \end{cases}$$

- We can implement the factorial in C++ with an iterative approach
- ...But we can also use recursion.

Recursion

```
1 unsigned int factorial = 1;
2 for (unsigned int i = n; i > 0; i --) {
3     factorial = factorial * i;
4 }
5
6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }
```

- You may have already seen recursion in mathematical definitions. E.g., the factorial:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot ((n-1)!) & \text{otherwise} \end{cases}$$

- We can implement the factorial in C++ with an iterative approach
- ...But we can also use recursion.
 - Fact directly returns a value only when $n = 0$ (base case)
 - Otherwise, Fact invokes itself passing $n-1$
 - Eventually, the base case is reached
- Note that the implementation is closer to the mathematical definition.

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Since $n=3$ is not equal to 0, Fact(3) will invoke Fact(2)
 - The execution of Fact(3) is paused until Fact(2) returns

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Since $n=2$ is not equal to 0, Fact(2) will invoke Fact(1)
 - The execution of Fact(2) is paused until Fact(1) returns

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Since $n=1$ is not equal to 0, Fact(1) will invoke Fact(0)
 - The execution of Fact(1) is paused until Fact(0) returns

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??
Fact(1)	0x1040	n = 1
	0x1044	r = ??

Executing Fact(3)

```

6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }

```

- Since $n = 0$ is the base case, Fact(0) will return 1

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??
Fact(1)	0x1040	n = 1
	0x1044	r = ??
Fact(0)	0x1060	n = 0
	0x1064	r = 1

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Fact(1) will then resume execution, multiplying the output of Fact(0) with n and returning the result

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??
Fact(1)	0x1040	n = 1
	0x1044	r = 1

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Fact(2) will then resume execution, multiplying the output of Fact(1) with n and returning the result

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = 2

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Fact(3) will then resume execution, multiplying the output of Fact(2) with n and returning the result

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = 6

Recursion

- When using recursion, we must ensure that:
 1. there are one or more base cases
 - i.e., smallest problems for which we return a solution, without recursion
 2. every recursion step reduces to a smaller problem
 3. every sequence of recursion steps eventually reaches one of the base cases
- In the Factorial example: the base case is $n = 0$, we recursively call Fact on smaller and smaller numbers, so we eventually reach the base case $n = 0$
- If any of the 3 conditions above is not satisfied, recursion may not work properly!
 - Risk of non-termination or crashes (stack overflow)
- When writing a recursive function, we need to assume that each recursive call with a smaller problem computes the correct solution
 - Example: to write Fact(n), we need to assume that Fact($n-1$) is correct
- Assuming that a recursive call works correctly is called the “recursive leap of faith”

Recursion: rules of thumb

- Identify the base cases (i.e., the “smallest” cases whose solution does not need recursion)
- Solve the base cases
- Address the recursive cases (i.e., the non-base cases that need recursive calls)
 - Ensure that the arguments to the recursive calls are “smaller” than the original arguments
 - Ensure that recursive calls eventually reach one of the base cases

Complexity

- We are often interested in estimating the resources needed by an algorithm:
 - time: number of operations
 - space: amount of memory/disk
- Such resources usually depend on the size of the algorithm inputs (denoted N)
- Complexity of a problem: given a concrete problem (e.g., sorting a list of numbers) what time/space resources are needed by the best solution algorithm?
- Notes:
 - Some problems are not computable! (i.e., no algorithm exists)
 - Sometimes we have a trade-off between time and space
 - For many problems, the precise complexity is not known:
 - we may know some solution algorithms, but we don't know whether a better one exists
 - we can give a lower bound to the complexity

Complexity

- We are often interested in the worst-case time / space requirements of an algorithm, given an input size N . In such cases, we use Big-O notation. E.g.:
 - $2N^2 + 17N + 53$ operations $\rightarrow O(N^2)$ time complexity
- We only consider dominant terms. This is because, as N grows,
 - larger exponents have more impact
 - constant factors and minor terms tend to become irrelevant
- For example:
 1. A good algorithm, time: $3000N \rightarrow O(N)$ time complexity
 2. A bad algorithm, time: $2N^2 \rightarrow O(N^2)$ time complexity
- Above some N , the algorithm (1) performs better (less operations) than (2)

More examples of complexity (see lecture code)

- Efficient search: binary search
 - Naive search (linear search) of an element in a set takes $O(n)$
 - Binary search is a divide-and-conquer $O(\log n)$ solution
- Efficient sorting: merge sort
 - The recursion paradigm directly triggers an efficient solution!
 - Naive bubble sort: $O(n^2)$ for array of size n
 - Merge sort: $O(n \log n)$ (theoretical optimum)
- Efficient exponentiation in cryptography ($a^n \bmod p$)
 - Naive exponentiation: $O(n)$
 - Efficient exponentiation: $O(\log n)$
 - Efficient solution is hard to program without recursion!

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 9
 - Ask questions if something is unclear (including previous assignments)

DTU

